CSE-4301
Object Oriented Programming
2023-2024

Week-3

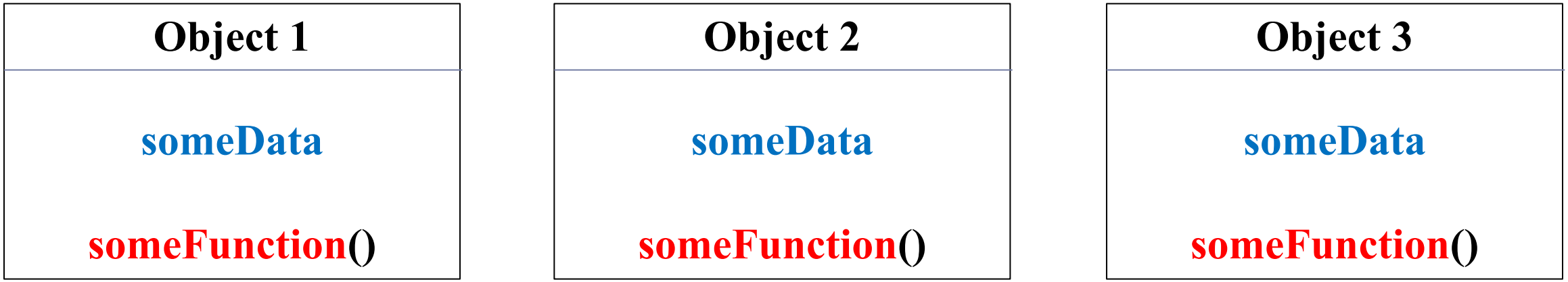
Objects and Classes

-Faisal Husain
Assistant Professor, Department Of Computer Science And Engineering, IUT

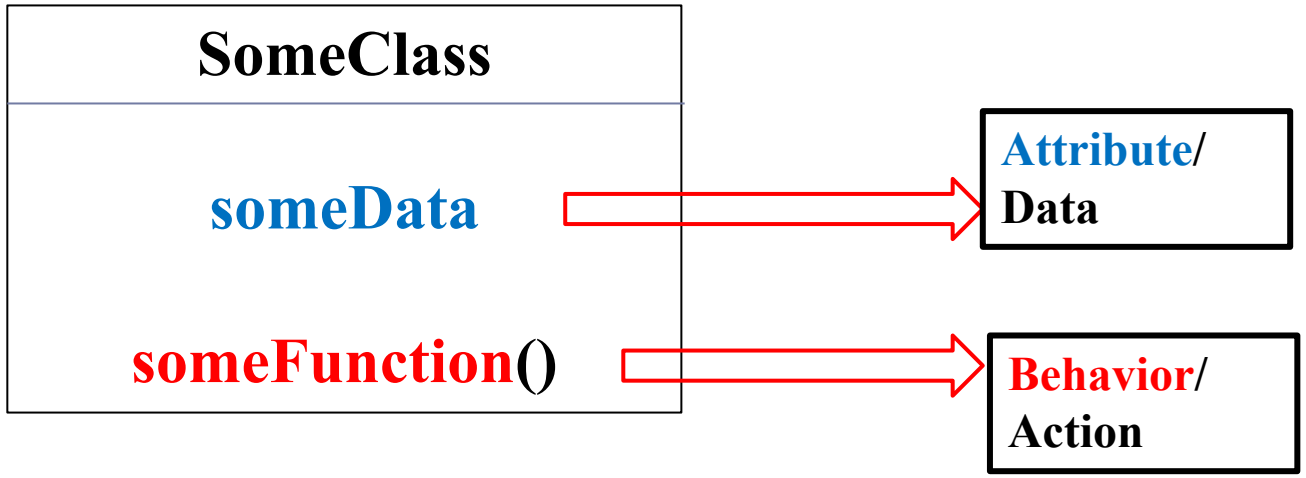
Outline

- Simple class definition
- Accessing Member data
- Calling Member functions
- Data Hiding
- Use of Member functions
- Role of Class Implementation Programmer vs Client Code Programmer
- Constructor, Initializer List, Destructor, Copy Constructor
- Pass an object to a function and return an object
- Member Functions Defined Outside the Class
- **static** class data
- **const** member function
- **const** object

Example Object



Example Object

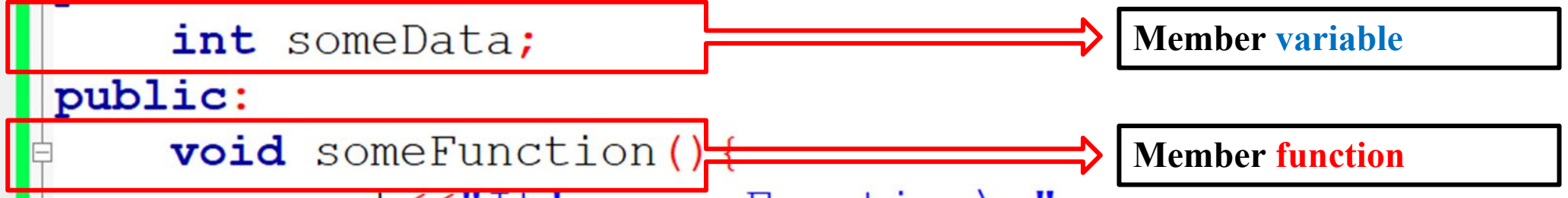


Simple Class Definition

```
1  #include<iostream>
2  using namespace std;
3
4  class SomeClass{
5  private:
6      int someData;
7  public:
8      void someFunction(){
9          cout<<"It's someFunction\n";
10     }
11 };
```

Simple Class Definition

```
1   #include<iostream>
2   using namespace std;
3
4   class SomeClass{
5   private:
6       int someData;
7   public:
8       void someFunction(){
9           cout<<"It's someFunction\n";
10      }
11  };
```



Simple Class Definition

```
1   #include<iostream>
2   using namespace std;
3
4   class SomeClass{
5   private:
6       int someData;
7   public:
8       void someFunction(){
9           cout<<"It's someFunction\n";
10      }
11  };
```

Keyword : class

Name of the class: SomeClass

Simple Class Definition

```
1   #include<iostream>
2   using namespace std;
3
4   class SomeClass{
5   private:
6       int someData;
7   public:
8       void someFunction(){
9           cout<<"It's someFunction\n";
10      }
11  };
```

Access Specifier

Data hiding feature

Simple Class Definition

```
1   #include<iostream>
2   using namespace std;
3
4   class SomeClass{
5   private:
6       int someData;
7   public:
8       void someFunction(){
9           cout<<"It's someFunction\n";
10      }
11  };
```

Private members can only be accessed from **within the class**.

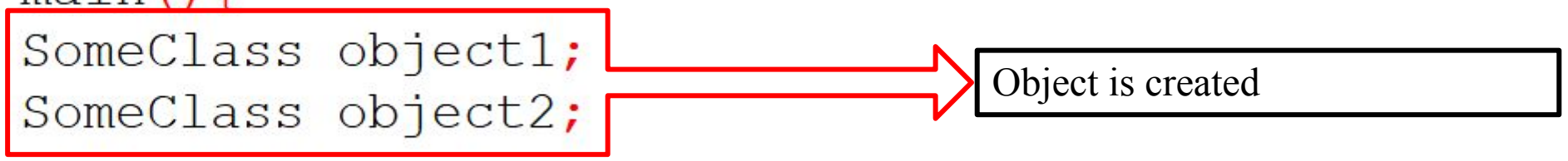
Simple Class Definition

```
1   #include<iostream>
2   using namespace std;
3
4   class SomeClass{
5   private:
6       int someData;
7   public:
8       void someFunction(){
9           cout<<"It's someFunction\n";
10      }
11  };
```

Public members can be accessed from **within or outside the class**.

Defining Object

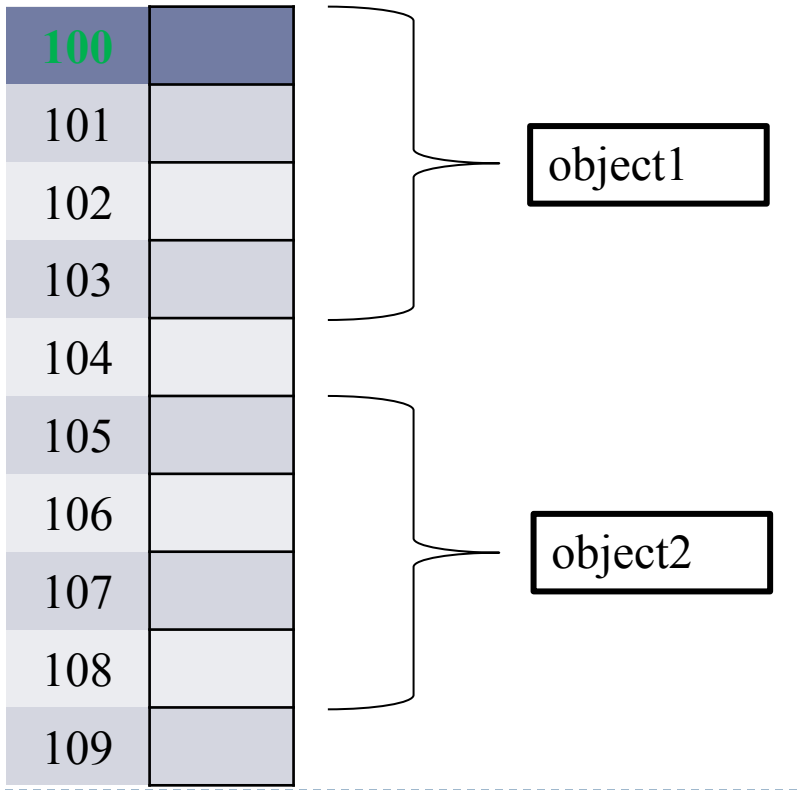
```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16  }
```



Defining Object

```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16  }
```

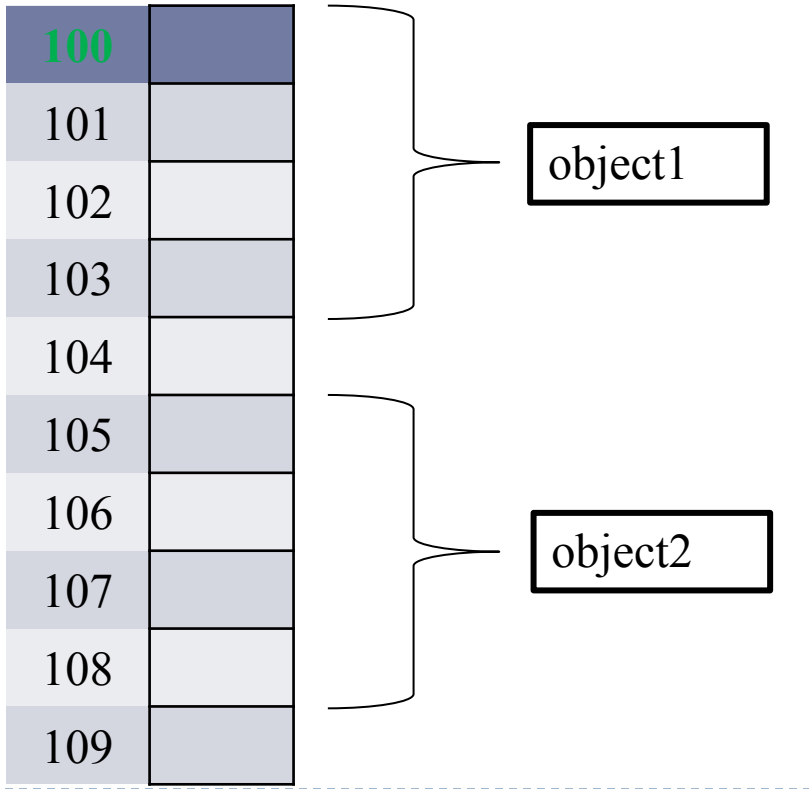
Object is created



Defining Object

```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16  }
```

Object is created



For each object, separate memory is allocated.

Access member of an object

```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16      object1.someFunction();
17  }
18
```

Using Dot(.) operator you can access a member of an object.

public member.
can access outside the class definition.

Access member of an object

```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16      cout<<object1.someData;
17  }
```

private member.
Cannot access outside the class definition.

Access member of an object

```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16      cout<<object1.someData;
17  }
```

private member.
Cannot access outside the class definition.

Generally, data members should be declared private and member functions should be declared public.

Access member of an object

```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16      cout<<object1.someData;
17  }
```

private member.
Cannot access outside the class definition.

Generally, data members should be declared private and member functions should be declared public.

As the data member is private, the change of data is only possible by the member function. It limits the scope of debugging.

Access member of an object

```
13  int main(){
14      SomeClass object1;
15      SomeClass object2;
16      cout<<object1.someData;
17  }
```

private member.
Cannot access outside the class definition.

Generally, data members should be declared private and member functions should be declared public.

As the data member is private, the change of data is only possible by the member function. It limits the scope of debugging.

Check the practice code for class definition.

Use of Getter Setter Function

Change/ Modification/ Refactoring in the code is normal phenomena in the software development process.

Use of Getter Setter Function

Change/ Modification/ Refactoring in the code is normal phenomena in the software development process.

Best programming practice localize the effect of change of data member using get set function to access and manipulate the data.

Use of Getter Setter Function

Change/ Modification/ Refactoring in the code is normal phenomena in the software development process.

Best programming practice localize the effect of change of data member using get set function to access and manipulate the data.

```
void setSomeData(int value){
    someData = value;
}

int getSomeData(){
    return someData;
}
```

```
///Instead of manipulating data directly
object.someData = 5;
///Change with a function
object.setSomeData(5);
```

Use of Getter Setter Function

Change/ Modification/ Refactoring in the code is normal phenomena in the software development process.

Best programming practice localize the effect of change of data member using get set function to access and manipulate the data.

```
void setSomeData(int value){
    if(value>0) someData = value;
    else someData = 0;
}
int getSomeData(){
    return someData;
}
```

```
///Instead of manipulating data directly
object.someData = 5;
///Change with a function
object.setSomeData(5);
```

Role of Class Implementation Programmer vs Client Code Programmer

```
class Counter
{
private:
    int count;
public:
    void reset(){
        count = 0;
    }
    void inc_count(){
        count++;
    }
    int getCount(){
        return count;
    }
};
```

Role of Class Implementation Programmer vs Client Code Programmer

```
class Counter
{
private:
    int count;
public:
    void reset(){
        count = 0;
    }
    void inc_count(){
        count++;
    }
    int getCount(){
        return count;
    }
};
```

Class Developer

Role of Class Implementation Programmer vs Client Code Programmer

```
class Counter
{
private:
    int count;
public:
    void reset(){
        count = 0;
    }
    void inc_count(){
        count++;
    }
    int getCount(){
        return count;
    }
};
```

Class Developer

```
int main()
{
    Counter c1;
    c1.reset();
    c1.inc_count();
    c1.inc_count();
    c1.inc_count();
    cout<<c1.getCount();
    return 0;
}
```

Application Developer

Constructor

- Member function executed automatically whenever an object is created.
  - *Same name as the class name*
  - *No return*
  - *May have zero or more argument*

```
Counter()
{
    count=0;
}
```

```
Counter(int value)
{
    count=value;
}
```

Constructor

- Member function executed automatically whenever an object is created.
  - *Same name as the class name*
  - *No return*
  - *May have zero or more argument*

- If no constructor is **present** in the class definition compiler will add a default constructor. Default constructor has zero parameter with empty function body.

check the practice code for constructor
(zero or more argument)

Initializer list

- One can use initializer list in the constructor to initialize the member variables.

```
Counter():count(0)
{}

Counter(int value):count(value)
{}
```

- Initializer list is the only way to initialize ***const*** member variable and reference.

check the practice code initializer list

Destructor

- Member function executed automatically whenever an object is ***destroyed*** {when an object is destroyed?}.
  - *Same name as the class name with tilde (~) sign before the name*
  - *No return*
  - *No argument*

```
~Counter(){
    cout<<"Destructor Function\n";
}
```

- Necessity will be clearer in future topics

Copy Constructor

- classname object1(argument_list);
- classname object2(object1);
- classname object3=object1;

Copy Constructor

- classname object1(argument_list);
- classname object2(object1);
- classname object3=object1;

```
Counter c1(10);
Counter c2 = c1;
Counter c3(c2);
```

Copy Constructor

- classname object1(argument_list);
- classname object2(object1);
- classname object3=object1;

If no copy constructor is defined for a class, then **Default Copy Constructor** will be added by the compiler.

Default Copy Constructor does a member-wise copy.

Object as Function Argument & Return Objects from Function

check the Distance class example

Member Functions Defined Outside the Class

- The class definition must include the declarations of all its member functions.
- The implementation (definition) of any number of member functions can be written separately (even in separate file).
- The separate parts (class definition and its member function implementations) are compiled independently.
- The linker will then connect these pieces together during the build process.
- This approach hides the implementation details of the member functions from the class definition.

Member Functions Defined Outside the Class

```
class Distance
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)
    {  }
    Distance(int ft, float in)  : feet(ft), inches(in)
    {  }
    void getdist()
    {
        cout << "\nEnter feet: ";
        cin >> feet;
        cout << "Enter inches: ";
        cin >> inches;
    }
    void showdist()
    {
        cout << feet << "\'-" << inches << '\"';
    }

    void add_dist(Distance,  Distance);
};
```

```
void Distance::add_dist(Distance d2,  Distance d3)
{
    inches = d2.inches + d3.inches;
    feet = 0;
    if(inches >= 12.0)
    {
        inches -= 12.0;
        feet++;
    }
    feet += d2.feet + d3.feet;
}
```

Member Functions Defined Outside the Class

```
class Distance
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)
    {  }
    Distance(int ft, float in)  : feet(ft), inches(in)
    {  }
    void getdist()
    {
        cout << "\nEnter feet: ";
        cin >> feet;
        cout << "Enter inches: ";
        cin >> inches;
    }
    void showdist()
    {
        cout << feet << "\'-" << inches << '\"';
    }

    void add_dist(Distance,  Distance);
};
```

```
void Distance::add_dist(Distance d2,  Distance d3)
{
    inches = d2.inches + d3.inches;
    feet = 0;
    if(inches >= 12.0)
    {
        inches -= 12.0;
        feet++;
    }
    feet += d2.feet + d3.feet;
}
```

This is only the declaration.
The implementation of **add_dist()** is not included in the class definition of Distance.

Member Functions Defined Outside the Class

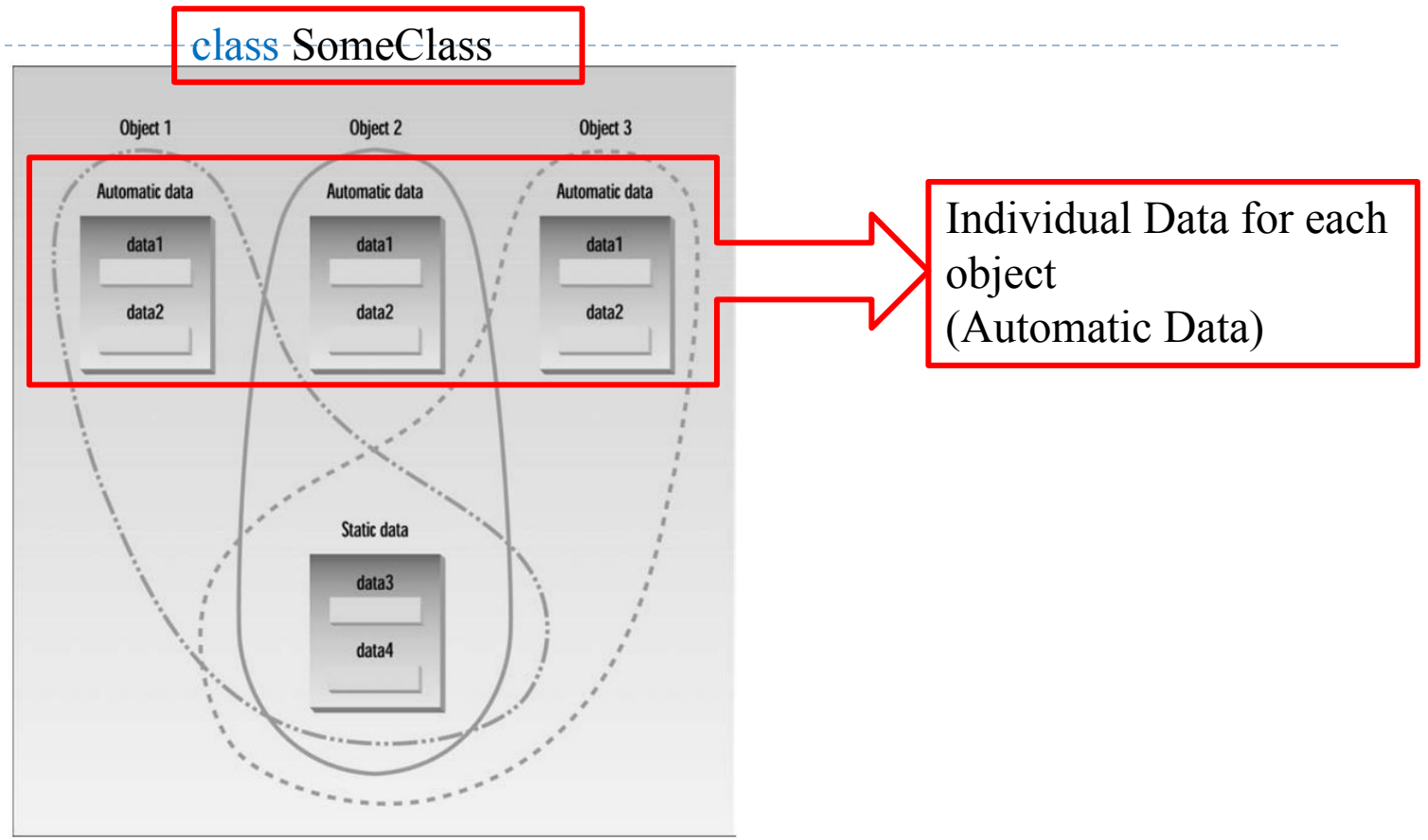
```
class Distance
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)
    {  }
    Distance(int ft, float in)  : feet(ft), inches(in)
    {  }
    void getdist()
    {
        cout << "\nEnter feet: ";
        cin >> feet;
        cout << "Enter inches: ";
        cin >> inches;
    }
    void showdist()
    {
        cout << feet << "\'-" << inches << '\"';
    }

    void add_dist(Distance,  Distance);
};
```

```
void Distance::add_dist(Distance d2,  Distance d3)
{
    inches = d2.inches + d3.inches;
    feet = 0;
    if(inches >= 12.0)
    {
        inches -= 12.0;
        feet++;
    }
    feet += d2.feet + d3.feet;
}
```

The implementation of **add_dist()** is outside the class definition of Distance.
You must use class name and scope resolution operator (::) before the class name to indicate this function to be a member of the said class.

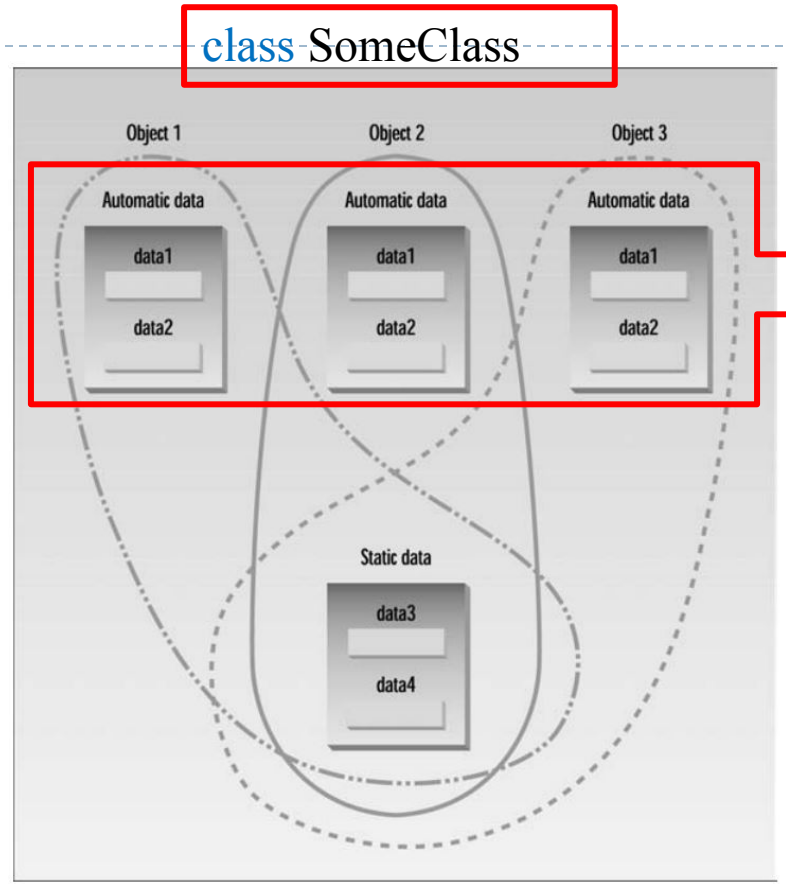
Automatic class data

- **auto** storage class specifier is default storage class.



Automatic class data

- **auto** storage class specifier is default storage class.
- Each object will have separate memory for its automatic member data.

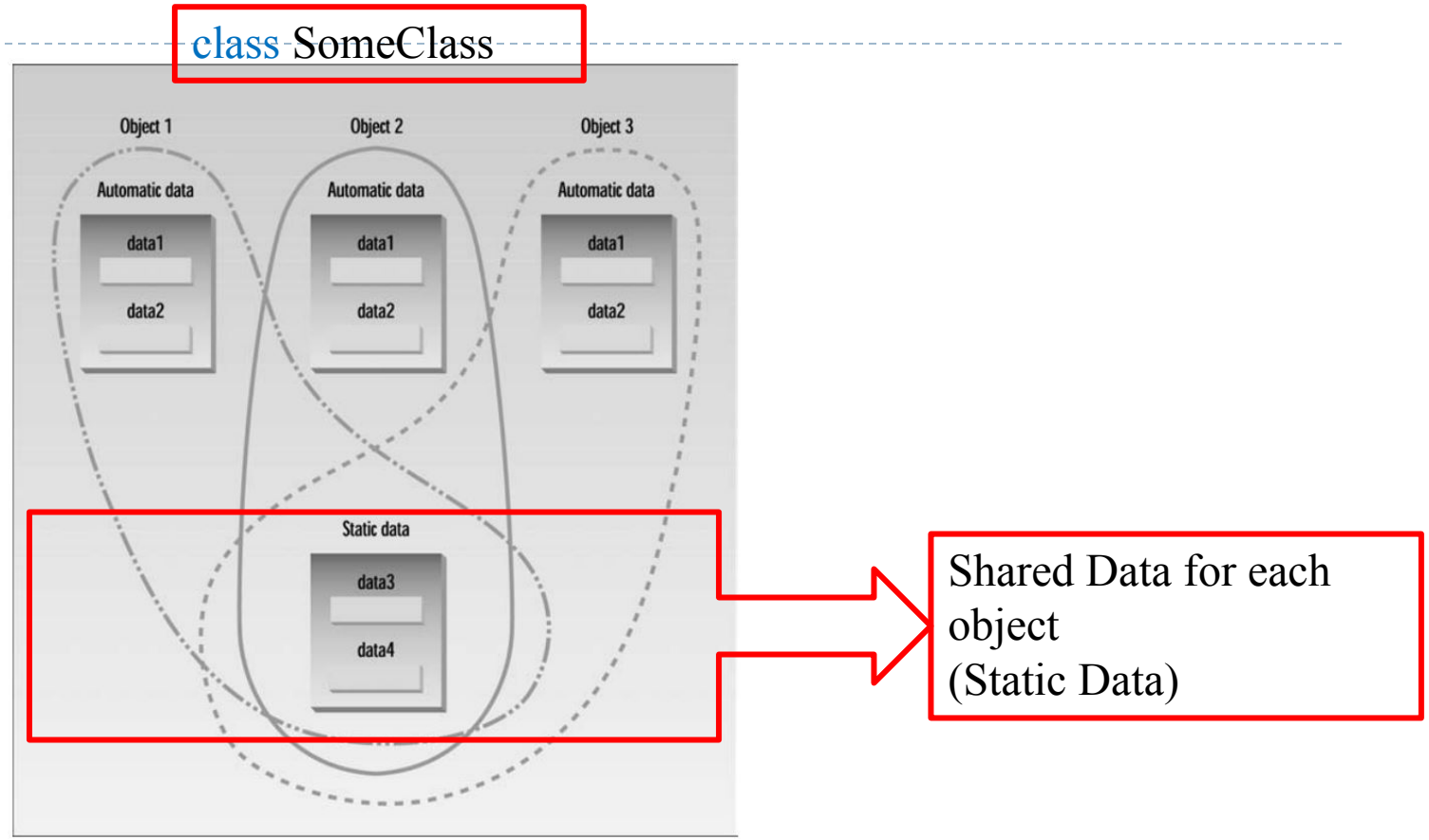


Individual Data for each object (Automatic Data)

Object1.data1,
Object2.data1,
Object3.data1
are the different variables.

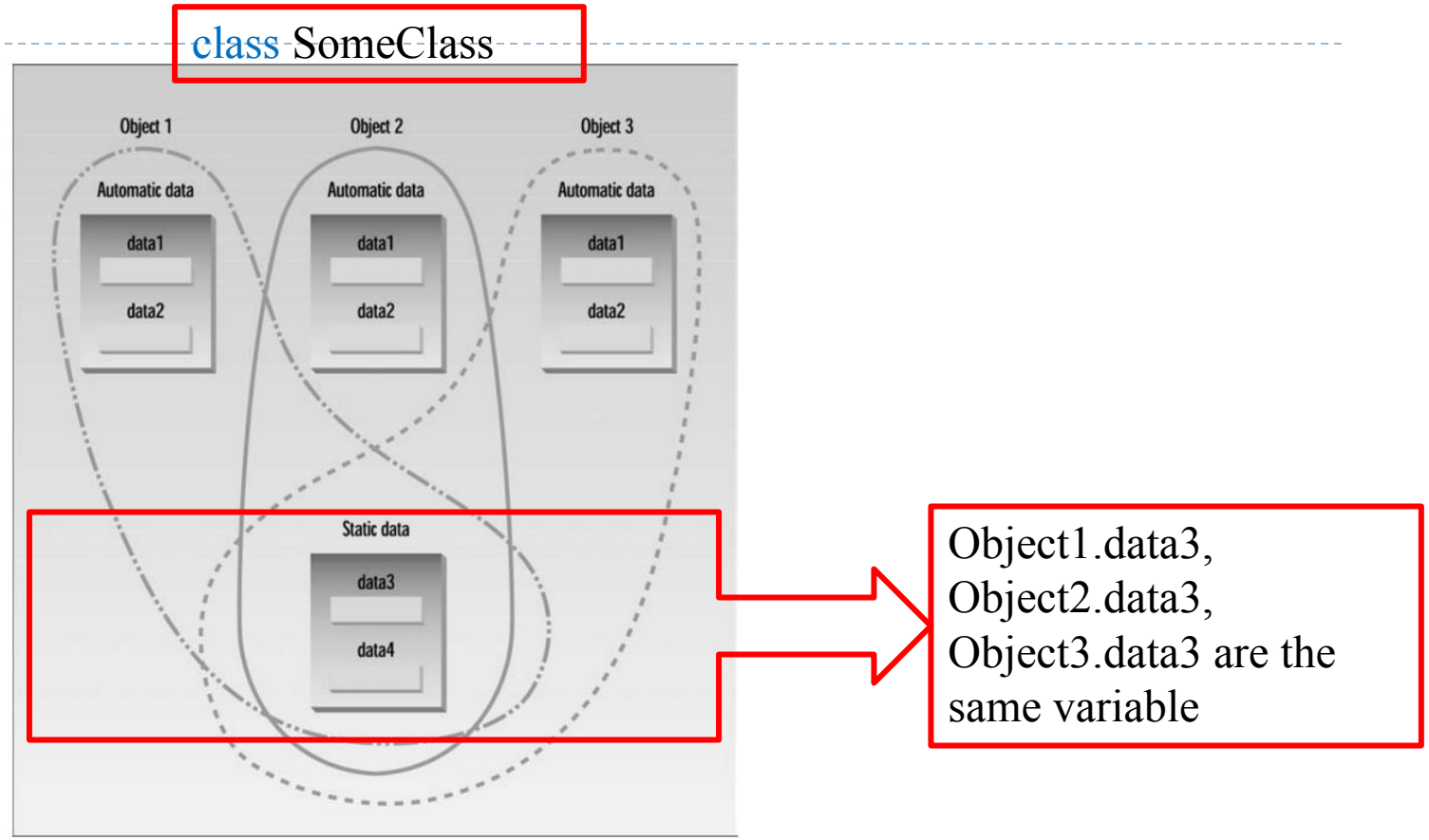
static class data

- Static class data is a single attribute shared by all the objects.



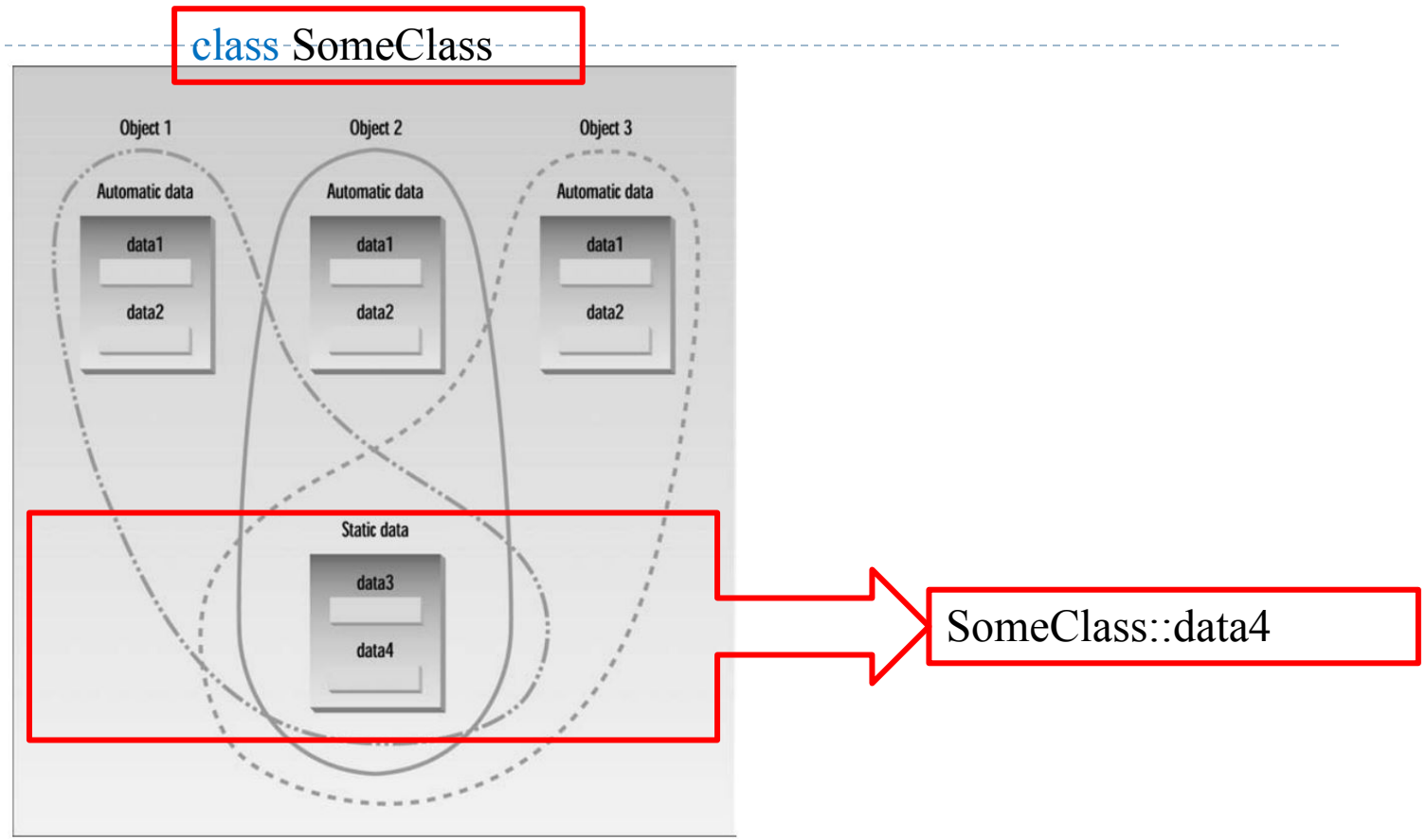
static class data

- Static class data is a single attribute shared by all the object
- Every object will see the same value of the static class data.



static class data

- Static class data is a single attribute shared by all the object
- Every object will see the same value of the static class data.
- Static data is also known as data of the class. Thus, it can be access without creating any object.



const Member Function

- In a **const** member function
  - no member data can be modified
  - only other **const** member function can be called.

```
class Distance                          //English Distance class
{
private:
    int feet;
    float inches;
public:                                 //constructor (no args)
    Distance() : feet(0), inches(0.0)
    {  }                                //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    {  }
    void getdist()                      //get length from user
    {
    void dummy()
    {
        cout << "Dummy\n";
    }
    void showdist() const               //display distance
    {
        cout << feet << "\'-" << inches << '\"';
    }

    Distance add_dist(const Distance&) const;      //add
};
```

const Member Function

- In a **const** member function
  - no member data can be modified
  - only other **const** member function can be called.

Add **const** keyword after the first parenthesis.

```
class Distance                          //English Distance class
{
private:
    int feet;
    float inches;
public:                                 //constructor (no args)
    Distance() : feet(0), inches(0.0)
    {  }                                //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    {  }
    void getdist()                      //get length from user
    {
    void dummy()
    {
        cout << "Dummy\n";
    }
    void showdist() const               //display distance
    {
        cout << feet << "\'-" << inches << '\"';
    }

    Distance add_dist(const Distance&) const;      //add
};
```

const Member Function

```
23      void dummy()
24      {
25          cout << "Dummy\n";
26      }
27      void showdist() const           //display distance
28      {
29          dummy();
30          cout << feet << "\'-" << inches << '\"';
31      }
```

In **dummy()** no member variable is modified.

const Member Function

```
23      void dummy()
24      {
25          cout << "Dummy\n";
26      }
27      void showdist() const           //display distance
28      {
29          dummy();
30          cout << feet << "\'-" << inches << '\"';
31      }
```

Still,
dummy() is not a **const** member function.
So cannot be called in const member function.

only other **const** member function can be called.

const Member Function

```
void dummy() const
{
    cout << "Dummy\n";
}
void showdist() const           //display distance
{
    dummy();
    cout << feet << "\'-" << inches << '\"';
}
```

Making **dummy()** a **const** member function will solve this issue.

const function argument in member function

- Like **const** function argument (cannot change the value of the arguments which are marked as **const**)

```
33      void Distance::test_const_function_argument(const int value)
34      {
35          value = 5;
36      }
```

const object

- Cannot modify a **const** object
- Can call only **const** member function
- Only constructor function is called while creating the object

```
class Distance                          //English Distance class
{
private:
    int feet;
    float inches;
public:                                 //constructor (no args)
    Distance() : feet(0), inches(0.0)
    {  }                                //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    {  }

    void getdist()                      //get length from user
    {
        cout << "\nEnter feet: ";
        cin >> feet;
        cout << "Enter inches: ";
        cin >> inches;
    }
    void showdist() const               //display distance
    {
        cout << feet << "\'-" << inches << '\"';
    }

    Distance add_dist(const Distance&) const;     //add
};
```

```
int main()
{
    const Distance Football(300,0);
    Football.showdist();
}
```

const object

- Cannot modify a **const** object
- Can call only **const** member function
- Only constructor function is called while creating the object

```
class Distance                              //English Distance class
{
private:
    int feet;
    float inches;
public:                                     //constructor (no args)
    Distance() : feet(0), inches(0.0)
    {  }                                    //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    {  }

    void getdist()                          //get length from user
    {
        cout << "\nEnter feet: ";
        cin >> feet;
        cout << "Enter inches: ";
        cin >> inches;
    }
    void showdist() const                   //display distance
    {
        cout << feet << "\'-" << inches << '\"';
    }

    Distance add_dist(const Distance&) const;      //add
};
```

```
int main()
{
    const Distance Football(300,0);
    Football.showdist();
}
```

const object

- Cannot modify a **const** object
- Can call only **const** member function
- Only constructor function is called while creating the object

```
class Distance                          //English Distance class
{
private:
    int feet;
    float inches;
public:                                 //constructor (no args)
    Distance() : feet(0), inches(0.0)
    {  }                                //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    {  }

    void getdist()                      //get length from user
    {
        cout << "\nEnter feet: ";
        cin >> feet;
        cout << "Enter inches: ";
        cin >> inches;
    }
    void showdist() const               //display distance
    {
        cout << feet << "\'-" << inches << '\"';
    }

    Distance add_dist(const Distance&) const;      //add
};
```

```
int main()
{
    const Distance Football(300,0);
    Football.showdist();
}
```

Reading Assignment

- **Chapter-6 Object and classes (Page- 216-257)**
  - **Object-Oriented Programming in C++ [Fourth Edition] -Robert Lafore**